AdPilot v2

Build Brief for Claude Code · Advanced Edition

Grounded · Explainable · Self-critiquing · Learns from real results · Measured

Repo: github.com/SHREE167/adpilot (work locally on branch v2, no push)

Live v1: adpilot-olive-nine.vercel.app

Stack: Next.js 15 App Router · TypeScript · XGBoost (trees walked in TS) · Vercel · Postgres (optional)

Prepared for: Sai · Builder: Claude Code (local, laptop) · 10 October 2026

1. TL;DR

The problem

- Plans feel generic. Two different snack shops in Chennai with ₹3,000 get nearly the same plan, because every number comes from a model trained on 12,000 *synthetic* briefs plus a static rate card.
- The app knows nothing real about the user's website, competitors, timing, audience size or local creators, and it never checks whether spending is even worth it.

The fix: four layers

- **Grounding**: live APIs (PageSpeed, Places, Meta reach, YouTube, festival calendar, search trends) so every plan cites real local facts.
- **Intelligence**: reality check (break-even), exact TreeSHAP, a concrete launch kit, zero-rupee layer, critic agent.
- **Interaction + learning**: budget slider, calibrated forecast ranges, actuals feedback loop, Thompson-sampling test budget.
- **Proof**: agent trace, an evaluation harness that *measures* specificity and hallucination, ad previews and a shareable plan.

Positioning: *The honest media planner for India's smallest businesses: grounded in live local data, explainable, self-critiquing, and it tells you when not to spend.*

Non-negotiables

- No ad buying, no user OAuth to Meta/Google. API keys live on the server only.
- Zero-key mode works exactly as today. Every new source is optional and degrades gracefully.
- Every number on screen is labelled **LIVE · source · date** or **ESTIMATE**. Never present an estimate as live data.
- A plan always returns, even if every external API fails or times out.
- **Local only:** commit on branch v2; never push or touch GitHub until Sai says "push now".

2. Current state (v1)

File	Role today
lib/agent.ts	runAgent(): scrape → recognize → XGBoost → allocate → forecast → copy. Internal 14s budget.
lib/scrape.ts	Fetches the domain (4s timeout): title, meta, H1, OG, JSON-LD, excerpt.
lib/recognize.ts, lib/llm.ts	Heuristic product intel + optional LLM refine. LLM chain: Groq → OpenAI → Gemini.
lib/scoring.ts	XGBoost per channel + policy rules (LinkedIn < ₹15k, score < 40 cutoff).
lib/ml/infer.ts, features.ts	Walks exported XGBoost trees in TS. 25 features, 12 channels. explainScore() = zero-ablation, not SHAP.
ml/train.py, ml/model.json (840 KB)	Synthetic training data. Nodes have feature/threshold/yes/no/leaf only, **no cover counts**.
lib/allocate.ts, lib/forecast.ts	Pure functions: 2–5 channels, 50% cap, rupee split; impressions/clicks/reach/conversions + weekly rollup.
lib/placements.ts, copy-templates.ts	Placement picks and template copy (LLM fills when a key exists).
lib/storage.ts, rate-card.ts	Postgres (adpilot_plans, adpilot_rate_card) or memory + localStorage. Static 2026 India rate card.
app/actions.ts, app/page.tsx	Server Action createPlan → runAgent. page.tsx sets maxDuration = 60.

File	Role today
components/*	Glass/gold UI (Fraunces/Syne, anime.js v4), Recharts, compare view, rate-card editor.

Weaknesses v2 must fix

- Synthetic training data: the model explains the generator's rules, not the market. State it openly; F13 + F20 are the answer.
- Explanations are zero-ablation guesses, not true SHAP.
- Static rate card, no seasonality, no audience-size ceiling, single-point forecasts.
- No awareness of site quality, tracking, competitors, creators, or break-even economics.
- No way to *prove* plans are specific or that the LLM is not inventing numbers.

3. Target architecture

Pipeline v2 (each step writes a trace span, see F15):

```
1  scrape              (+ tracking & funnel detection)
2  recognize product
3  ENRICH              parallel tools, each optional, hard timeout, cached
     ├─ siteReadiness     tracking checks (PageSpeed deferred)
     ├─ localMarket       Google Places
     ├─ calendar          static festival JSON
     ├─ audience          Meta reach estimate
     ├─ creators          YouTube Data API
     └─ searchDemand      Google Trends via SerpAPI (optional)
4  XGBoost score       + exact TreeSHAP values
5  SIGNAL ADJUST       season multipliers, reach caps, calibrated rate card (F13)
6  allocate            + explore slice (F20, only when actuals exist)
7  forecast            + calibrated low/mid/high ranges
8  REALITY CHECK       break-even verdict
9  CRITIC              one review round, structured, allow-listed fixes
10 LAUNCH KIT + copy   LLM grounded only on signals, numbers post-validated
```

Time budget

- Core path (1, 2, 4, 6, 7, 8) under ~10s with zero keys.
- Enrichment via **Promise.allSettled**; per-source timeout (default 5s). Timeout = status **timeout**, never an error page.
- **PageSpeed is slow** (often 10–30s): deferred enrichment requested by the dashboard after render, cached 24h.
- page.tsx already sets maxDuration = 60. Raise the internal budget from 14s to ~25s only after checking the Vercel plan limit.
- Critic and LLM steps are skipped automatically if the remaining budget is too small (recorded in the trace).

Signal contract

```
// lib/enrich/signal.ts
export type SignalStatus = "ok" | "skipped" | "timeout" | "error";
export interface Signal<T> {
  source: string;        // "google_places", "meta_reach", ...
  status: SignalStatus;
  fetchedAt: string;     // ISO, original fetch time (kept on cache hits)
  ms: number;
  cached: boolean;
  data?: T;
  reason?: string;       // "no API key", "HTTP 403", ...
}
export interface EnrichmentBundle {
  siteReadiness: Signal<SiteReadiness>;
  pageSpeed?:    Signal<PageSpeedResult>;   // filled later (deferred)
  localMarket:   Signal<LocalMarket>;
  calendar:      Signal<SeasonWindow>;
  audience:      Signal<AudienceEstimate>;
  creators:      Signal<CreatorPick[]>;
```

```
  searchDemand:  Signal<SearchDemand>;
}
// PlanResult gains OPTIONAL fields so old saved plans still render:
//   enrichment?, realityCheck?, shap?, launchKit?, critic?, ranges?,
//   zeroRupee?, trace?, budgetGrid?, explore?, planVersion: 2
```

Caching, mocks, provenance

- Table **adpilot_cache** (key text PK, value jsonb, expires_at timestamptz); in-memory Map fallback. TTLs: PageSpeed 24h, Places/Meta/YouTube/Trends 7d.
- **ADPILOT_MOCK_APIS=1**: every source reads fixtures/api/<source>/*.json. Demo, tests and eval run fully offline.
- UI badges: **LIVE · Google Places · 4 Oct**, **ESTIMATE**, muted **Not connected**. A 'Data sources' footer lists every signal with status and ms.

4. Feature specs

P0 kills the generic feel · **P1** intelligence · **P2** interaction and learning · **P3** proof and polish (advanced).

P0 — Grounding

F1 · Site Readiness Check [P0]

What the user sees	'Ready to spend?' card, 0–100 score with fixes: mobile load time, Meta Pixel / GA4 / GTM, HTTPS, viewport, WhatsApp click-to-chat, visible phone/CTA, og:image.
How it works	Part A (no key): detect fbq(/ fbevents.js, gtag(/ googletagmanager.com, GTM- IDs, wa.me and api.whatsapp.com links, tel: links, viewport meta. GTM-only → 'GTM found; a pixel may be inside it' (never a false 'no pixel'). Part B (key, deferred): PageSpeed Insights v5, strategy=mobile → score + LCP. Rules: no pixel → warn before Meta spend; LCP > 4s → fix first; tiny budget + weak site → route to a WhatsApp/DM funnel.
Files	lib/scrape.ts, lib/enrich/site-readiness.ts, lib/enrich/pagespeed.ts, app/actions.ts, components/ReadinessCard.tsx
Done when	Tracking checks work with zero keys; PageSpeed fills in after render; plan never waits on it.

F2 · Local Market Scan [P0]

What the user sees	'18 similar businesses near you · avg rating 4.1 · top 5 names' and 'Not on Google Maps → create a free Business Profile'.
How it works	Places API (New) Text Search '{category keyword} in {city}' + a search for the business name. Strict field masks (rating fields raise the SKU tier), 7-day cache. Density feeds copy angles, the zero-rupee layer and the store_visits goal.
Files	lib/enrich/places.ts, components/LocalMarketCard.tsx
Done when	Real names/counts with LIVE badge; skipped cleanly without a key; quota cap set in Google Cloud.

F3 · Festival & Season Calendar [P0]

What the user sees	'Pongal on 14 Jan (in 12 days): launch this week. Festival-week CPMs typically rise (ESTIMATE +25%).'
How it works	data/festivals-in.json (2026–2027): name, date, states, categories, cpmMultiplier, advice, verified flag. data/city-state.json maps city → state. Weekly forecast applies multipliers to overlapping weeks. Calendarific optional cross-check.
Files	data/festivals-in.json, data/city-state.json, lib/enrich/calendar.ts, lib/forecast.ts, components/TimingCard.tsx
Done when	Regional festivals differ for Chennai vs Bengaluru vs Delhi; weekly chart shows the bump; works offline.

F4 · Real Audience Size [P0]

What the user sees	'Chennai · 18–24 · snacks, street food → reach pool ~1.2M–1.4M (LIVE · Meta)'. Facebook/Instagram reach never exceeds the pool.
How it works	Server-side system-user token. Resolve city via search type=adgeolocation, interests via type=adinterest; call the ad account's delivery/reach estimate → MAU bounds. API version pinned in env. Exact targeting spec shown in the launch kit. Verify endpoint names against current Meta docs.
Files	lib/enrich/meta-reach.ts, lib/forecast.ts (reach cap), components/AudienceCard.tsx
Done when	Bounds with LIVE badge; reach capped; skipped without token.

F5 · Local Creator Finder [P0]

What the user sees	5 real YouTube creators: name, subscribers, link, why they fit, collab cost band (ESTIMATE).
How it works	YouTube Data API v3: search.list type=channel, q='{category} {city}', regionCode=IN, relevanceLanguage from form → channels.list statistics → keep 2k–200k subs. search.list costs 100 of 10,000 daily units: cache hard. No Instagram creator discovery promised.
Files	lib/enrich/youtube-creators.ts, components/CreatorsCard.tsx
Done when	Real channels with links; cached; skipped without key.

F6 · Search Demand [P0-optional]

What the user sees	'Interest in "healthy snacks" in Tamil Nadu is rising; rising queries: …'
How it works	Google Trends via SerpAPI (geo e.g. IN-TN) → trend + rising queries → keyword themes in placements.ts. Never fabricate search volumes.
Files	lib/enrich/search-demand.ts, lib/placements.ts
Done when	Trend shown with LIVE badge or hidden; no invented volumes anywhere.

P1 — Intelligence

F7 · Launch Kit [P1]

What the user sees	Per channel: exact targeting, days/times, creative count + format, keywords, first-week checklist, the one metric to watch. e.g. 'Instagram Reels · ₹1,800 · Chennai 18–24 · 3 Reels Fri–Sun 6–10pm · Pongal week · install Pixel first'.
How it works	Built deterministically from signals, then the LLM rewrites for tone. **Grounding guard:** extract every number from LLM output; any number not present in the plan or EnrichmentBundle is removed and logged to the trace. Template fallback without a key.
Files	lib/launch-kit.ts, lib/grounding-guard.ts, lib/llm.ts, components/LaunchKit.tsx
Done when	Two businesses in the same city get visibly different kits; zero unguarded numbers.

F8 · Reality Check (break-even) [P1]

What the user sees	Verdict: Healthy / Tight / Likely loss. 'You need 47 sales to break even → 3.1% conversion needed; typical ~1%. Start with a ₹1,000 test or fix free channels first.'
How it works	Optional form fields price (₹) + margin %. breakEvenUnits = budget / (price × margin); requiredCvr = units / clicks (low/mid/high). Compare to goal benchmark bands in data/benchmarks.json (ESTIMATE). Skip if fields empty.
Files	lib/reality-check.ts, components/PlanForm.tsx, components/RealityCard.tsx, data/benchmarks.json
Done when	Unit-tested math; clear verdict copy; never blocks the plan.

F9 · Exact TreeSHAP [P1]

What the user sees	Per-channel waterfall: 'Instagram 82 = base 51 + age 18–24 (+14) + food (+9) + Tamil (+6) + budget (+2)'.
How it works	Exact TreeSHAP (Lundberg et al. 2018) in lib/ml/shap.ts over the tree JSON. **Blocker:** nodes lack cover. ml/train.py exports per-node cover from the booster dump, re-export model.json, self-check still passes. Human labels for the 25 features. Replaces zero-ablation in explainScore().
Files	ml/train.py, ml/model.json, lib/ml/shap.ts, lib/ml/infer.ts, components/ShapWaterfall.tsx
Done when	Unit test: base + Σ SHAP = prediction within 1e-3 for every channel on all self-check rows.

F10 · Zero-Rupee Layer [P1]

What the user sees	'Before you spend anything' list above the paid mix: Google Business Profile (if F2 says unlisted), WhatsApp Business catalog + Status, organic Reels, local groups.
How it works	Rule-based ranking from signals (readiness, market, category, goal, budget). Each item has a 'why'.
Files	lib/zero-rupee.ts, components/ZeroRupeeCard.tsx
Done when	Items change with signals; no generic filler.

F12 · Critic Agent [P1]

What the user sees	'Reviewed by critic: 2 changes' collapsible list, e.g. 'LinkedIn ₹1,200 is below a viable test → moved to Search.'
How it works	Second LLM pass over plan JSON + signals → schema-validated issues [{severity, channel, issue, fix}]. Only allow-listed fixes applied deterministically (drop channel, shift %, add warning). Max one round; deterministic rule-critic fallback without an LLM key.
Files	lib/critic.ts, components/CriticPanel.tsx
Done when	Plan still sums to budget after fixes; every applied fix is in the trace.

P2 — Interaction and learning

F11 · Budget Slider [P2]

What the user sees	Drag 0.25×–4× budget; mix, forecast, reality verdict update instantly.
How it works	log_budget is a model feature, so the server precomputes scores at ~10 log-spaced budgets (budgetGrid) and returns them with the plan; the client re-runs allocate + forecast (pure) and snaps to the nearest point. Signals reused, never re-fetched. Model never shipped to the browser.
Files	lib/budget-grid.ts, components/BudgetSlider.tsx
Done when	< 50 ms per move; totals always equal the slider budget.

F13 · Actuals Feedback Loop [P2]

What the user sees	'Add real results' on a saved plan (spend, impressions, clicks, conversions per channel) → Estimate vs Actual chart + MAPE.
How it works	Table adpilot_actuals. Recalibrate per (channel, city tier, category) with Bayesian shrinkage: new = (n0·prior + n·observed)/(n0 + n). Future plans for that segment use the calibrated card; the dashboard shows 'calibrated from N real campaigns'.
Files	lib/actuals.ts, lib/storage.ts, app/plans/[id]/results/page.tsx, components/ActualsChart.tsx
Done when	Entering results visibly changes later plans for that segment, with the reason shown.

F14 · Calibrated Forecast Ranges [P2]

What the user sees	Low / mid / high everywhere a single number appeared (e.g. reach 42k–61k).
How it works	Default band ±30%. Once ≥ 5 actuals exist for a channel, use **empirical (split-conformal style) intervals**: the 10th/90th percentile of actual/estimate ratios. Narrower when LIVE signals back the number (Meta pool).
Files	lib/forecast.ts, lib/intervals.ts, components/Charts.tsx
Done when	Ranges everywhere; band source labelled (default vs calibrated).

F20 · Explore Budget (Thompson sampling) [P2-advanced]

What the user sees	'10% test slice → Telegram: we're unsure about it for your segment; this tests it cheaply.'
How it works	Beta posterior per (channel, segment) on conversion rate from actuals; sample once per plan; route a capped 5–15% slice to the channel with the highest sampled value not already in the mix. Off until ≥ 3 actuals exist. Ties to the Chapelle & Li paper in the report.
Files	lib/explore.ts, lib/allocate.ts
Done when	Deterministic with a seed in tests; never exceeds the cap; clearly labelled as a test.

P3 — Proof and polish (advanced)

F15 · Agent Trace & Observability [P3]

What the user sees	'How this plan was made' tab: timeline of every step with ms, status, inputs/outputs summary, cache hits, LLM provider, tokens, guard removals, critic fixes.
How it works	lib/trace.ts: span(name, fn) wrapper records start/end/status/meta into PlanResult.trace. Every pipeline step and tool call is wrapped. Optional: export as JSON. This is the agent's observability layer and doubles as debugging.
Files	lib/trace.ts, lib/agent.ts, app/plans/[id]/trace/page.tsx, components/TraceTimeline.tsx
Done when	Every plan has a complete trace; failed/timeout tools visible; total ms matches agentMs.

F16 · Evaluation Harness [P3]

What the user sees	**npm run eval** prints a scorecard and writes eval/report.md: specificity, diversity, grounding, latency, critic impact.
How it works	eval/golden-briefs.json: 20 varied briefs (city, category, budget, goal, language). Runs the full pipeline in mock mode and computes: **Specificity** = grounded facts cited per plan; **Diversity** = mean Jaccard distance of channel mixes + launch-kit text across briefs; **Grounding** = % of numbers traceable to plan/signals (target 100%); **Signature check** = ₹3k vs ₹40k mixes differ; **Latency** p50/p95; **Critic delta** = issues before/after. Compare v1 vs v2 in one table.
Files	eval/golden-briefs.json, eval/run.ts, eval/metrics.ts, package.json script
Done when	Report shows v2 beats v1 on specificity and diversity with 100% grounding.

F17 · Ad Previews (vernacular) [P3]

What the user sees	Each ad angle rendered as an Instagram post frame, a WhatsApp message bubble and a Google search ad, in the chosen language (Tamil/Hindi/English).
How it works	Pure React/CSS mockups (no brand logos), copy from the launch kit. Character limits enforced per format with a warning.
Files	components/AdPreview.tsx
Done when	Previews match format limits; work on mobile width.

F18 · Shareable Plan + WhatsApp Summary [P3]

What the user sees	'Download one-page plan' (print-friendly page) and 'Copy WhatsApp summary' (short text with mix, budget, top 3 actions).
How it works	app/plans/[id]/print/page.tsx with print CSS; lib/share-text.ts. No new dependency.
Files	app/plans/[id]/print/page.tsx, lib/share-text.ts
Done when	Prints cleanly to one or two A4 pages; summary under 700 characters.

5. Engineering rules for Claude Code

- Zero-key mode must keep working exactly as today. Check after every feature.
- Every external call: server-only, timeout, try/catch, cache, returns a Signal, wrapped in a trace span. Never throw out of an enrichment.
- Keys only from process.env on the server. Never import lib/enrich/* into a client component.
- New PlanResult fields optional; old saved plans (localStorage + Postgres) must still render. Add planVersion: 2.
- No new npm dependency without asking. Prefer fetch + small helpers. Unit tests may use node:test (built in).
- UI: use the frontend-design skill; extend the existing glass/gold look; respect prefers-reduced-motion; mobile width first.
- After each feature: npm run lint, npx tsc --noEmit, npm run build, unit tests pass. One local commit per feature.
- **Git:** branch v2 only. Never git push / pull / gh / any remote, never merge to main, until Sai says "push now".
- Keep README roadmap and .env.example in sync with every new env var.

6. What Sai must set up externally

Claude Code cannot create these. All optional for the app; each unlocks its feature. M1 and M3 need none of them.

#	Service	Unlocks	Where / how	Cost	Env vars
1	Postgres (Neon / Vercel Storage)	Cache, actuals, F13/F20	Vercel → Storage → Postgres, or neon.tech; copy URL to .env.local	Free tier	POSTGRES_URL
2	Groq API key	Launch kit wording, critic	console.groq.com	Free tier	GROQ_API_KEY
3	Google Cloud project + PageSpeed Insights API	F1 part B	console.cloud.google.com → enable API → API key	Free	PAGESPEED_API_KEY
4	Places API (New)	F2	Same project; billing account needed. Set daily quota + budget alert	Free cap per SKU, then paid	GOOGLE_PLACES_API_KEY
5	YouTube Data API v3	F5	Same project → enable → key	Free, 10k units/day	YOUTUBE_API_KEY
6	Meta developer app + Business account + ad account	F4	developers.facebook.com (Business app); system-user token with ads_read; no spend needed	Free	META_ACCESS_TOKEN, META_AD_ACCOUNT_ID, META_API_VERSION
7	SerpAPI (optional)	F6	serpapi.com	Limited free tier	SERPAPI_API_KEY
8	Calendarific (optional)	F3 cross-check	calendarific.com	Free tier	CALENDARIFIC_API_KEY

Manual work

- Restrict every Google key to its API. Keys go in .env.local only (never committed); later in Vercel env vars.
- Verify festival dates in data/festivals-in.json (they shift every year) and flip verified: true.
- Collect 3–5 real campaign results (a friend's shop, a college event page, your own ₹500 test) to demo F13/F14/F20 honestly.
- Once keys work, record one fixture set per API so demos and eval run offline.

7. Milestones

#	Scope	Needs keys	Est.
M1	Enrichment framework (Signal, timeouts, cache, mock mode), F15 trace spans, provenance badges, F1 part A, F3 calendar	None	2–3 days
M2	F1 part B (PageSpeed, deferred), F2 Places, F10 zero-rupee	#3, #4	2 days
M3	F8 reality check, F9 TreeSHAP (retrain with cover), F16 eval harness v0 (v1 baseline)	None	3 days
M4	F4 Meta reach, F5 YouTube, F7 launch kit + grounding guard	#2, #5, #6	3–4 days
M5	F11 budget slider, F14 ranges (default band)	None	2 days
M6	F12 critic, F13 actuals loop, F14 calibrated intervals, F20 explore slice, (F6)	#1, #2, (#7)	4 days
M7	F17 ad previews, F18 share/print, F15 trace page UI	None	2 days
M8	Eval v2 vs v1 report, fixtures, tests, README, demo script, report/PPT material	—	2 days

Each milestone ships alone. Stopping after M3 still gives a clearly better, measurable project.

8. Testing and demo

- **Unit** (node:test): allocation sums; SHAP additivity; break-even math; timeout → 'timeout'; cache TTL; grounding guard; shrinkage; Thompson cap.
- **Mock mode**: ADPILOT_MOCK_APIS=1 runs the whole pipeline offline.
- **E2E** (webapp-testing skill, Playwright): zero-key flow, mock flow, old saved plan renders, slider updates totals, trace page loads.
- **Eval**: npm run eval → eval/report.md with v1 vs v2 table. Put this table in the project report.
- **Demo script**: ₹3,000 Chennai snack brand → readiness warnings → Pongal timing → competitors + Maps → creators → reality verdict → SHAP waterfall → critic changes → slider → trace → add actuals → rate card + ranges move → eval scorecard.

9. Risks and honest limits

Risk	Mitigation
Model trained on synthetic data	Say so in the UI model card and report; F13/F20 recalibrate with real results.
API approvals take days (Meta)	Everything optional; build against mock fixtures first.
Places API can bill	Strict field masks, 7-day cache, daily quota, budget alert.
Vercel timeouts	Per-source timeouts, allSettled, deferred PageSpeed, budget-aware skipping.
GTM-injected pixels invisible to static scraping	'GTM found; pixel may be inside' instead of a false negative.
LLM invents numbers	Grounded prompt + grounding guard + eval grounding metric.
Few actuals → noisy calibration	Shrinkage toward prior; intervals stay default until ≥ 5 points; explore off until ≥ 3.

Risk	Mitigation
Festival multipliers / benchmarks are guesses	Labelled ESTIMATE, editable JSON, refined by actuals.
Privacy	Only public business/creator data; no personal data stored from scraped sites.

10. Claude Code setup (laptop, Windows) + prompts

```
git clone https://github.com/SHREE167/adpilot
cd adpilot
git checkout -b v2
npm install
copy .env.example .env.local
mkdir docs        (put this PDF at docs\ADPILOT_V2_BRIEF.pdf)
irm https://claude.ai/install.ps1 | iex      (PowerShell; needs Git for Windows)
claude
```

- Inside Claude Code: `/plugin marketplace add anthropics/skills`, then install the plugin containing frontend-design, webapp-testing and skill-creator from `/plugin`.
- Press **Shift+Tab** until 'plan mode' shows, then paste the starting prompt.
- One milestone per session; `/clear` between milestones. CLAUDE.md + this PDF carry context.

Starting prompt (Milestone 1)

```
You are working on AdPilot (Next.js 15 App Router + TypeScript).
docs/ADPILOT_V2_BRIEF.pdf is the source of truth. Read ALL of it first.

GIT RULES (strict, permanent)
- Work only on local branch v2. One local commit per feature.
- NEVER run git push, git pull, git fetch, gh, or anything that
  touches a remote or GitHub until I say exactly "push now".
- Never merge into main.

STEP 0 - Skills
- List your skills. Use frontend-design for every UI change (extend
  the glass/gold Fraunces+Syne look, do not redesign). Use
  webapp-testing to verify flows in a browser. Use skill-creator at
  the end. If any is missing, tell me and stop.

STEP 1 - Understand (plan mode, no edits)
- Read README.md, lib/agent.ts, lib/types.ts, lib/scrape.ts,
  lib/scoring.ts, lib/ml/infer.ts, lib/ml/features.ts,
  lib/allocate.ts, lib/forecast.ts, lib/storage.ts, app/actions.ts,
  app/page.tsx, components/Dashboard.tsx, ml/train.py.
- Confirm ml/model.json nodes have no "cover" field (needed for F9).
- Draft CLAUDE.md: summary, commands, file map, section 5 rules,
  the GIT RULES above, and the milestone table from section 7.
- Give me a plan for MILESTONE 1 ONLY: files, data contracts, test
  plan, risks. Wait for my OK.

STEP 2 - Build Milestone 1 (after I approve)
- lib/enrich/: signal.ts (Signal<T>, withTimeout, runEnrichment via
  Promise.allSettled), cache.ts (Postgres adpilot_cache with TTL +
  in-memory fallback), mock.ts (ADPILOT_MOCK_APIS=1 reads
  fixtures/api/<source>/*.json).
- F15 core: lib/trace.ts span() wrapper; wrap every runAgent step;
  store PlanResult.trace.
- F1 part A: tracking + funnel detection in scrape.ts ->
  lib/enrich/site-readiness.ts.
- F3: data/festivals-in.json (2026-2027, verified:false),
  data/city-state.json, lib/enrich/calendar.ts, weekly multipliers
  in lib/forecast.ts.
- Wire enrichment as step 3 in runAgent. Add optional enrichment,
  trace and planVersion: 2 to PlanResult. Old plans must render.
- Dashboard: "Ready to spend?" card, "Timing" card,
  LIVE / ESTIMATE / Not connected badges, "Data sources" footer.
- Unit tests with node:test for signal timeout, cache TTL, calendar.

RULES
- Zero-key mode works exactly as today.
- No new npm dependency without asking me.
- Keys only via process.env on the server.
- Every number in the UI is LIVE (source + date) or ESTIMATE.
- npm run lint, npx tsc --noEmit, npm run build, tests must pass
  before each commit.

WHEN DONE
- Use skill-creator to write .claude/skills/adpilot-enrichment/
  SKILL.md: how to add a data source (Signal, timeout, cache key,
```

```
  mock fixture, trace span, badge, tests). Commit locally.
- Stop and tell me: what changed, how to test with npm run dev,
  and exactly what I must set up before Milestone 2.
```

Next-milestone prompt

```
Read CLAUDE.md, docs/ADPILOT_V2_BRIEF.pdf (sections 3-9) and the
adpilot-enrichment skill. Build MILESTONE <N> only.
Plan first and wait for my OK. Follow every rule in section 5,
including the GIT RULES (local commits only, no push).
Use frontend-design for UI and webapp-testing to verify.
Run npm run eval if it exists and show the score change.
Stop at the end with: changes, how to test, and setup needed for
the next milestone.
```